Planning & Reinforcement Learning

UniVR - Dipartimento di Informatica

Tommaso Toffali

1° Semestre 2026/2027

Indice

Parte I
Classical Planning

1 Introduction to Planning

1.1 Historical Context and Motivation

Planning has been a central topic in artificial intelligence since the inception of the field at the **Dartmouth Conference in 1956**, where the foundings fathers of AI first gathered to define the scope and ambitions of this new discipline. Planning represent one of the foundamental capabilities required for intelligence behavior: the ability to reason about sequences of actions that achieve desired goals.

The motivation for studying automated planning are manifold:

- **Autonomous agents**: Enabling robots, software agents, and autonomouse system to make decision and act independently in complex environemnts
- **Resource optimization**: Finding optimal or near-optimal stragies for Resource allocation, scheduling, and logistics
- **Scientific applications**: Modeling and solving problems in domain such as space exploration, manufacturing, and healtcare
- **Theoretical foundations**: Understanding the computional complexity and formal properties of reasoning about action and change

1.2 The planning Cycle

The complete planning process involves several interconnected pahses that form a continuos cycle:

1. **Plan Generation**: Given a description of the current state, available actions, and desired goals, synthesize a sequence of actions (a plan) that transforms the inital state into a goal state. This phase involves:
    - Analyzing the initial state to understand what is currently true
    - Identifying the gap between the current state and the goal state
    - Searching throungh the space of possible action sequences
    - Evaluating candidate plans for correctness and optimality
    - Selecting the best plan according to specified criteria (e.g., shortes length, minimum cost, fastest execution)
2. **Plan Deployment**: Execute the generated plan in the actual environment, monitoring the execution to detect deviations from expected behavior. This phase includes:
    - Translating abstract plan actions into concrete executable commands
    - Monitoring sensors and feedback to verify that actions have the expected effects

- Detecting discrepancies between predicted and actual states
- Maintaining a record of executed actions for debugging and learning

3. **Replanning**: When execution monitoring detects that the current plan is no longer valid (due to unexpected events, actions failures, or environmental changes), generate a new plan or repair the existing one. Replanning strategies includes:
   - **Plan repair**: Modify the existing plan minimally to accomodate the new situation
   - **Replan from scratch**: Generate an entirely new plan from the current state
   - **Contingency planning**: Use pre-computed alternative plans for anticipated failures

This cycle reflects the reality that planning systems must operate in dynamic, partially unpredictable environments where initial plans may need adaptation. The cyle continues iteratively until the goal is achived or deemed unreachable.

1.2.1 Challenges in Real-World Planning

Real-World deployment of planning systems faces several challengs:

- **Excution uncertainty**: Actions may fail or have unexpected outcomes
- **Incomplete information**: The planner may not have complete knowledge of the environment
- **Dynamic environments**: The world may change while the plan is beign executed
- **Computional constraints**: Planning must often occur in real-time with limited computational resoruces
- **Plan quality vs. planning time**: Trade-off between finding optimal plans and responding quickly

These challenge motivate the development of robust planning algorihms that cna handle unvertainty, adapt to changes, and operate efficiently under resource constraints.

2 Formalization of the Planning Problem

2.1 State Transition Systems

The mathematical foundation of planning rests on the concept of state transition system, which provides a formal model of how actions transform states.

Definizione 2.1 (State Transition System)

state transition system is a tuple $\Sigma = (S, A, \gamma)$ where:

- S is a finite or countably infinite set of **states**

- A is a finite or countably infinite set of **actions**
- $\gamma : S \times A \to S$ is a **state transition function** that maps a state and an action to a resulting state

The transition function $\gamma(s, a) = s'$ specifies that executing action a in state s result in state s'. This function encodes the **dynamics** of the domain - how the world changes in response to actions.

2.1.1 Properties of State Transition System

State transition systems can be characterized by serveral important properties:

- **Determinism**: In a deterministic system, γ is a function - each state-action pair leads to exactly one successor state. In non-deterministic systems, multiple outcomes are possible.
- **Reachability**: A state s' is reachable from state s if there exists a sequence of actions that transform s into s'. The reachable state space from an initial state is often much smaller thant the totale state sapce.
- **Reversibility**: An action is reversible if there exists another action that undoes its effects. Many real-world actions are irreversible (e.g., breaking an object)
- **State space structure**: The connectivity and topology of the sate space significanyly affect planning complexity. Highly connected spaces may have many solution paths, while sparse spaces may have few or no solutions.

2.2 Classical Planning: Foundamental Assumptions

Classical planning makes several simplyfying assumptions that restrict the class of the problems considered but enable efficient algorithmic solutions.

Definizione 2.2 (Classical Planning Framework)

he classical planning framework relies on six key assumptions:

1. Finitely many states
2. Finitely many actions
3. Deterministic transition function
4. Full observability
5. Instantaneous actions
6. No exogenous events

1. **Finitely many states**: The state space S is finite, allowing exhaustive search techniques. This assumption ensures that the planning is decidable and that search alogirhms will terminate

   Jusitfication: While real-world domains may have infinite state spaces (e.g.

continuos variables), finite approximations are often sufficient for pratical purposes.

2. **Finitely many actions**: The action space A is finite, ensuring decidability. Each state has a finite branching factor in the state space graph.

   Jusification: Even in complex domains, the number of distinct action type is typically manageable, though the number of ground actions (instantieted with specific objects) may be LARGE

3. **Deterministic transition function**: For every state s and action a, there is exactlu one resulting state $\gamma(s, a)$. Actions have predictable, certain outcomes.

   Justification: Determinism simplified reasoning about action effects. Nondeterministic planning requires more complex formalism (e.g., Markov Decision Processes)

4. **Full observability**: The planner has complete knowledge of the current state at all times. There is no uncertainty about which state the system occupies.

   Justification: Full observability eliminates the need for belief state reasoning and sensing actions. Partial observability requires more sophisticated planning approaches.

5. **Instantaneous actions**: Actions have no duration, they occur instantaneously. Temporal reasoning is not required.

   Justification: Ignoring action durations simplifies the planning model. Temporal planning extends classical planning to handle durations and concurrent actions.

6. **No exogenous events**: Only the planning agent can change the world through deliberate actions. The environment remains static unless acted upon.

   Justification: Exogenous events (e.g., other agents, natural processes) introduce additional complexity. Classical planning assumes a static world between actions.

These assumptions, while restrective, capture a significant and important class of planning problems and provide a foundation for understanding more complex, realistic planning scenarios.

2.2.1 Relaxing Classical Assumptions

Modern planning research has devoloped extensions that relax these assumptions:

- **Probabilistic planning**: Handles non-deterministic actions with probability distributions over outcomes
- **Conformant planning**: Plans under partial observability without sensing

- **Contingent planning**: Generates conditional plans that include sensing actions
- **Temporal planning**: Reasons about actions durations and temporal constraints
- **Multi-agent planning**: Coordinates plans among multiple agents

2.3 The Planning Problem

Given a state transition system $\Sigma = (S, A, \gamma)$ we can now formally define the planning problem.

Definizione 2.3 (Planning Problem)

planning problem is a tuple $P = (\Sigma, s_0, G)$ where:

- $\Sigma = (S, A, \gamma)$ is a state transition system
- $s_0 \in S$ in the **initial state**
- $G \subseteq S$ is a set of **goal states**

Alternatively, goals can be specified as a **goal formula** g, a logical expression that is satisfied by exactly those states in G. This allows more compact and expressive goal specifications.

2.3.1 Goal specification

Goals can be specified in several ways:

- **Explicit goal states**: Enumerate the set G of accetable final states. This is impratical for large state spaces.
- **Goal formula**: A logical formula g such that $G = s \in S \mid s \models g$. For example, in propositional logic : $g = At(Robot, RoomB) \wedge Clean(RoomB)$
- **Goal conditions**: A set of propositions that must be true in the goal state. This is the most common approach in classical planning.
- **Utility functions**: In optimization settings, goals may be specified as utility functions to maximize or cost functions to minimize.

Definizione 2.4 (Solution Plan)

solution to a planning problem $P = (\Sigma, s_0, G)$ is a sequence of actions $\pi = (a_1, a_2, \ldots, a_n)$ such that:

$$s_n = \gamma(\gamma(\ldots\gamma(\gamma(s_0, a_1), a_2)\ldots, a_{n-1}), a_n) \in G$$

That is, executing the sequence of actions starting from the initial state s_0 results in a goal state.

We can also write this more compactly using the notation $\gamma^*(s, \pi)$ to denote the state reached by executing plan π from state s:

$$\gamma^*(s_0, \pi) \in G$$

2.3.2 Plan Quality

Not all solution plans are equally desirable. Common quality metrics include:

- **Plan length**: Number of actions in the plan. Shorter plans are often preferred.
- **Plan cost**: Sum of actions costs. Each action a may have an associated cost $c(a)$.
- **Makespan**: Total execution time (relevant in temporal planning).
- **Resource consumption**: Amount of resources used during execution.

An **optimal plan** minimizes the chosen quality metric among all solutions plans.

3 Representation of Planning Problems

3.1 The challenge of explicit representation

While the state transition system formalism is mathematically elegant, it faces a critical pratical challenge: **the curse of dimensionality**. Real-world planning domains can have exponentially large sate spaces. For example:

- A domain with n boolean variables has 2^n possible states
- A domain with k objects and m binary relations has up to 2^{mk^2} states
- Enumerating all states and transitions explicitly is infeansible for even moderately-sized problems

Esempio 3.1 (Blocks World Complexity)

onsider a Blocks World domain with n blocks. The number of possible configurations grows super-exponentially with n. For $n = 10$ blocks, there are more than 10^13 possible configurations. Explicitly representing the transition function would require storing information about trillions of state-action pairs.

This motivates the need for **compact, structured representation** that exploit regularities in the domain to represent large state spaces and transition functions implicitly.

3.2 Action Schemas: Structured Representation

The classical approach to compact representation uses **action schemas** (also called action template or operators). An action schema is a prametrized description of a family of related actions.

Definizione 3.2 (Action Schema)

n action schema consists of:

- **Name and parameters**: A symbolic name and typed parameters, e.g., $Move(x, y)$.
- **Preconditions**: A logical formula Pre specifying when the action can be executed
- **Effects**: A logical formula Eff specifying how the sate changes when the action in executed

Actions are **ground instances** of action schemas, obtained by binding the parameters to specific objects in the domain. For example, the schema $Move(x, y)$ might generate ground actions $Move(RoomA, RoomB)$, $Move(RoomB, RoomC)$, etc.

3.2.1 Advantages of Action Schemas

Action schemas provide severeal benefits:

- **Compactness**: A single schema can represent exponentially many ground actions
- **Generality**: Schemas capture the general structure of actions independent of specific objects
- **Scalability**: Adding new objects to the domain automatically generates new appliable actions
- **Knowledge reuse**: Schemas learned in one domain can be transferred to similar domains.

3.3 State Representation: Propositional logic

In classical planning, states are typically represented using **propositional logic**:

- A state is a set of **atoms** (propositional variables) that are true in that state
- Atoms not in the set are assumed false (**closed-world assumption**)
- Action preconditions and effects are logical formula over these atoms

The representation allows:

- Compact encoding of structured states
- Efficient reasoning about action applicability and effects
- Use of logical inference techniques for plan generation

3.3.1 State Update Semantics

Esempio 3.3 (STRIPS Assumption)

hen an action is executed, the state is updated according to the action's effects:

- **Add list**: Atoms that become true after the action
- **Delete list**: Atoms that become false after the action
- **Frame axioms**: Atoms not mentioned in the effects remain unchanged

The **STRIPS assumption** (named after the Stanford Research Institute Problem Solver) states that only atoms explicitly mentioned in the effects changem all other atoms persist unchanged. This simplifies reasoning about action effects.

3.4 Example: Blocks World

Consider the classic **Blocks World** domain, one of the most studied domain in planning research

3.4.1 Domain Description

The Blocks World consists of:

- A set of blocks that can be stacked on top of each other
- A table with unlimited space
- A robot arm that can pick up and put down blocks

Esempio 3.4 (Blocks World State Representation)

State predicates:

- $\texttt{On}(x, y)$: Block x is directly on top of blocks y.
- $\texttt{OnTable}(x)$: Block x is on the table
- $\texttt{Clear}(x)$: Block x has no blocks on top of it
- $\texttt{Holding}(x)$: The robot arm is holding block x.
- $\texttt{ArmEmpty}$: The robot arm is not holding anything

3.4.2 Action Schemas

PickUp(x): Pick up block x from the table

Parameters: x(block)
Precondition: $\texttt{Clear}(x) \land \texttt{OnTable}(x) \land \texttt{ArmEmpty}$
Effect: $\texttt{Holding}(x) \land \neg\texttt{OnTable}(x) \land \neg\texttt{Clear}(x) \land \neg\texttt{ArmEmpty}$

PutDown(x): Put down block x on the table

Parameters: x(block)
Preconditions: $\texttt{Holding}(x)$
Effect: $\texttt{OnTable}(x) \land \texttt{Clear}(x) \land \texttt{ArmEmpty} \land \neg\texttt{Holding}(x)$

Stack(x, y): Stack block x on top of block y.

Parameters: x, y(blocks)
Preconditions: $\texttt{Holding}(x) \land \texttt{Clear}(x)$
Effect: $\texttt{On}(x, y) \land \texttt{Clear}(x) \land \texttt{ArmEmpty} \land \neg\texttt{Holding}(x) \land \neg\texttt{Clear}(y)$

Unstack(x, y): Remove block x from on top of block y.

Parameters: x, y(block)
Precondition: $\texttt{Holding}(x) \land \texttt{Clear}(x) \land \texttt{ArmEmpty}$
Effect: $\texttt{Holding}(x) \land \texttt{Clear}(y) \land \neg\texttt{On}(x, y) \land \neg\texttt{Clear}(x) \land \neg\texttt{ArmEmpty}$

3.4.3 Example Problem Instance

Initial state: Blocks A, B, C are all on the table

$s_0 = \{\texttt{OnTable}(A), \texttt{OnTable}(B), \texttt{OnTable}(C), \texttt{Clear}(A), \texttt{Clear}(B), \texttt{Clear}(C), \texttt{ArmEmpty}\}$

Goal: Stack the blocks in the order A on B on C

$$G = \{\texttt{On}(A, B), \texttt{On}(B, C), \texttt{OnTable}(C)\}$$

Solution plan:

1. `PickUp`(B)
2. `Stack`(B,C)
3. `PickUp`(A)
4. `Stack`(A,B)

Parte II
Advanced Planning Representation

4 Reduction to Propositional Satisfiability

4.1 Motivation

We continue our discussion of the formalization and representation of planning problems. A fundamental technique in automated planning is to **reduce the planning problem to the stisfiability problem of propositional logic**. This reduction allows us to leverage powerful SAT solvers to find plans efficently.

Definizione 4.1 (Reduction to SAT)

he key idea is to encode:

- The initial state as a propositional formula
- The goal conditions as a propositional formula
- The action effects and preconditions as propositional formulas
- The constraint that actions must form a valid sequence

If the resulting formula is satisfiable, the satisfying assignment corresponds to a valid plan.

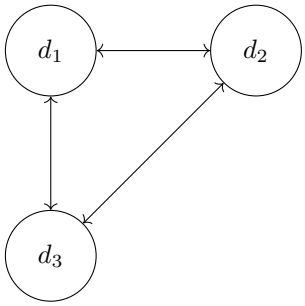
4.2 Example Domain: Dock Worker Robots (DWR)

To illustrate these concepts, we introduce the **Dock Worker Robots (DWR)** domain, a classic planning benchmark that models container logistics at a port.

4.2.1 Domain description

The DWR world consists of:

- **Docks**: Three docks d_1, d_2, d_3 arranged in a triangular configuration
- **Connectivity**: Docks are connected as follows: $d_1 \leftrightarrow d_3, d_1 \leftrightarrow d_2, d_2 \leftrightarrow d_3$.
- **Piles**: Storage locations at docks where containers can be stacked
- **Containers**: Object that need to be moved between locations
- **Robots**: Mobile agents that can move containers between piles



4.3 Initial State configuration

The initial state of our example problem is:

- **At dock** d_1: Pile p_1 contains two containers: c_1 (on top) and c_2 (below)
- **At dock** d_2:
  - Pile p_2 contains one container: c_3.
  - Pile p_3 is empty
- **At dock** d_3: Pile p_4 is empty

4.4 Formal Representation

4.4.1 Objects and Types

The domain contains objects of different **sorts** (types). Each object is represented by a constant symbol with an associated type:

Docks:

$$\text{Docks} = \{d_1, d_2, d_3\}$$

where each d_i is a constant symbol of sort `Dock`.

Containers:

$$\text{Containers} = \{c_1, c_2, c_3\}$$

where each c_i is a constant symbol of sort `Container`.

Robots:

$$\text{Robots} = \{r_1, r_2\}$$

where each r_i is a constant symbol of sort `Robot`.

Piles:

$$\text{Piles} = \{p_1, p_2, p_3, p_4\}$$

where each p_i is a constant symbol of sort `Pile`.

The complete set of objects is:

$$\text{Objects} = \text{Docks} \cup \text{Piles} \cup \text{Containers} \cup \text{Robots}$$

4.4.2 Static Relations

Some relations in the domain are **static** - they do not change during plan exectuion. These can be specified once and remain constant throughout.

Adjacency relation: Specifies which docks are directly connected, allowing robots to travel between them.

$$\texttt{adjacent} = \{(d_1, d_3), (d_3, d_1), (d_2, d_3), (d_3, d_2), (d_1, d_2), (d_2, d_1)\}$$

This realtion is symmetric: if dock d_i is adjacent to dock d_j, then d_j is adjacent to d_i.

Pile-Dock association: Each pile is permanently located at a specific dock.

$$\texttt{at}(p_1, d_1)$$
$$\texttt{at}(p_2, d_2)$$
$$\texttt{at}(p_3, d_2)$$
$$\texttt{at}(p_4, d_3)$$

4.4.3 Rigid Relations

The adjacency relation is defined over $\texttt{Docks} \times \texttt{Docks}$. This is called a **rigid relation** (also known as a **static relation**) - it doeas not change during plan execution. Actions cannot modify the physical layout of the docks or their connectivity.

Rigid relations are important because:

- They reduce the state space by factoring out unchanging aspects
- They can be checked once at planning time rather than during execution
- They simplify action precodinitions by providing fixed background knowledge

4.5 State Representation in DWR

4.5.1 What is a State?

A **state** is an assignment of values to propositional variables. More precisely, a state can be represented in two equivalent ways:

1. **First-order ground atoms**: Predicates where all arguments are constants (no variables)
2. **First-order ground terms**: Terms where all arguments are constants.

Important restriction: We only allow **flat expression**, meaning expressions with depth 1 (no nesting).

Esempio 4.2 (Forbidden Nesting)

Consider the predicate $\texttt{loc}(x, y)$ meaning "the location of x is y". We do **not** allow nested expressions such as:

$$\texttt{loc}(r_1, \texttt{loc}(r_2))$$

This would represent "the locaation of r_1 is the location of r_2", which violates the flatness constraints.

All atoms and terms must be flat - arguments of predicates and functions must be constants, not complex expressions.

4.5.2 State as Truth Assignment

A state assign trugth values to ground atoms. Consider describing the initial state s_0 of out DWR example:

$$\texttt{loc}(r_1, d_1)$$
$$\texttt{pos}(p_1, d_1)$$
$$\texttt{pos}(c_2, p_1)$$

These atoms being listed in the state means they are **implicitly assigned true**:

$$\begin{aligned} \texttt{loc}(r_1, d_1) &\leftarrow \text{true} \\ \texttt{pos}(p_1, d_1) &\leftarrow \text{true} \\ \texttt{pos}(c_2, p_1) &\leftarrow \text{true} \end{aligned}$$

Atoms not mentioned in the state are assumed false (closed-world assumption).

4.6 Two Representation Formalisms

There are two common approaches to representing planning problems, differing in how they encode state information.

4.6.1 Classical (Propositional) Representation

In the **classical representation**, states are represented using only **Boolean assignments** to propositional atoms. Each possible fact about the world is represented as a Boolean variable.

Example atoms:

- $\texttt{loc}(r_1, d_1)$: Robot r_1 is at dock d_1 (true/false)
- $\texttt{pos}(c_2, p_1)$: Container c_2 is at pile p_1 (true/false)
- $\texttt{empty}(r_1)$: Robot r_1 is not carrying anything (true/false)

Characteristics:

- Simple and uniform representation
- Direct correspondence to propositional logic
- Easy to encode as SAT problems
- May require many atoms for complex domains

4.6.2 State Variable Representation

The **state variable representation** uses a slightly different syntax based on **function symbols**. Instead of Boolean atoms, we use variables that can take on different values from a finite domain.

Example:

$$\texttt{loc}(r_1) = d_1$$

This means "the location of robot r_1 is d_1". Here, $\texttt{loc}(r_1)$ is a **state variable** (function that returns a value), and d_1 is its current value.

Comparison with classical representation:

Classical	State Variable
$\texttt{loc}(r_1, d_1) = \text{true}$	$\texttt{loc}(r_1) = d_1$
$\texttt{loc}(r_1, d_2) = \text{false}$	
$\texttt{loc}(r_1, d_3) = \text{false}$	

Advantages of state variables:

- More compact: one variable instead of multiple Boolean atoms
- Explicitly represents mutual exclusion (robot can only be at one location)
- Natural for domains with multi-value attributes
- Reduces the number of variables in SAT encodings

Relationship: The two representations are **equivalent in expressive power**. Any problem representable in one formalism can be translated to the other. The choice depends on:

- The planning algorithm being used
- The structure of the domain
- Efficiency considerations for the specific problem

4.6.3 Example: Complete State Representation

Let us represent the complete initial state s_0 of our DWR example in both formalisms.

Classical representation (partial):

$$\begin{aligned}
&\texttt{loc}(r_1, d_1) = \text{ true}\\
&\texttt{loc}(r_2, d_2) = \text{ true}\\
&\texttt{pos}(c_1, p_1) = \text{ true}\\
&\texttt{pos}(c_2, p_1) = \text{ true}\\
&\texttt{pos}(c_3, p_2) = \text{ true}\\
&\texttt{on}(c_1, c_2) = \text{ true}\\
&\texttt{emtpy}(r_1) = \text{ true}\\
&\texttt{empty}(r_2) = \text{ true}\\
&\texttt{top}(c_1, p_1) = \text{ true}\\
&\texttt{top}(c_3, p_2) = \text{ true}
\end{aligned}$$

State variable representation:

$$\begin{aligned}
&\texttt{loc}(r_1) = d_1 \\
&\texttt{loc}(r_2) = d_2 \\
&\texttt{pos}(c_1) = p_1 \\
&\texttt{pos}(c_2) = p_1 \\
&\texttt{pos}(c_3) = p_2 \\
&\texttt{on}(c_1) = c_2 \\
&\texttt{loaded}(r_1) = \text{ nil} \\
&\texttt{loaded}(r_2) = \text{ nil} \\
&\texttt{top}(p_1) = c_1 \\
&\texttt{top}(p_2) = c_3
\end{aligned}$$

Both representations capture the same information but organize it differently. The state variable representation is more compact and explicitly enforces constraints (e.g., a robot can only be at one location).

4.7 Key Insight: States as Boolean Assignments

The fundamental insight is that regardless of the representation formalism chosen:

A state is fundamentalyy a Boolean assignment

Wheter we use propositional atoms or state variables, we are ultimately assigning truth values (or equivalently, values from finite domains) to a set of variables.

5 Action Schemas in DWR

5.1 Structure of Action Schemas

An **action schema** α is a template for generating ground actions. It consists of:

Definizione 5.1 (Action Schema Components)

An action schema α has the following components:

- **Name and parameters**: $\alpha = \texttt{name}(z_1, z_2, \ldots, z_k)$ where z_i are typed parameters
- **Preconditions**: $\text{Pre}(\alpha) = \{p_1, p_2, \ldots, p_n\}$ where each p_i is a literal
- **Effects**: $\text{Eff}(\alpha) = \{q_1, q_2, \ldots, q_m\}$ where each q_i is a literal
- **Cost**: $c(\alpha) \in \mathbb{N}_0$ (typically a non-negative integer)

Important constraint: The parameters z_i of α can appear in the precodition literals p_j and effect literals q_j. The arguments in these literals can be either:

- **Constants**: Specific objects from the domain

- **Parameters**: Variables that will be instantiated

No arbitrary free variables are allowed - all variables must be parameters of the action schema.

Definizione 5.2 (Ground Action)

A **ground action** a is obtained by substituting all parameters z_i in action schema α with specific constants from the domain. The action a is an **instance** of α

5.2 Example: Take/Load Action

Consider the action of a robot taking a container from a pile. We define the action schema:

$$\alpha : \texttt{take}(r, c, c', p, d)$$

Parameters:

- r: robot (type `Robot`)
- c: container to take (type `Container`)
- c': container below c (type $\texttt{Container} \cup \{\texttt{nil}\}$)
- p: pile (type `Pile`)
- d: dock (type `Dock`)

5.2.1 Correcting Static Relations

Before defining the preconditions, we note that there are **two rigid (static) relations** in the DWR domain:

1. $\texttt{adjacent} : \texttt{Docks} \times \texttt{Docks}$ - specifies which docks are connected
2. $\texttt{at} : \texttt{Piles} \times \texttt{Docks}$ - specifies which pile is located at which dock

Both relations are rigid because the physical layout of the port does not change during plan execution.

5.2.2 Preconditions

For the `take` action to be executable, the following conditions must hold:

$$\text{Pre}(\texttt{take}(r, c, c', p, d)) = \left\{ \begin{array}{l} \texttt{loc}(r) = d \\ \texttt{at}(p, d) \\ \texttt{pos}(c) = p \\ \texttt{on}(c) = c' \\ \texttt{top}(p) = c \\ \texttt{cargo}(r) = \texttt{nil} \end{array} \right\}$$

Interpretation:

- The robot r is at dock d
- The pile p is located at dock d (static relation)
- Container c is positioned at pile p
- Container c is on top of container c' (or c' is `nil` if c is at the bottom)
- Container c is the top container of pile p
- The robot r is not carrying anything

5.2.3 Effects

When the `take` action is executed, the following changes occur:

$$\text{Eff}(\texttt{take}(r, c, c', p, d)) = \left\{ \begin{array}{l} \texttt{cargo}(r) = c \\ \texttt{top}(p) = c' \\ \texttt{pos}(c) = r \end{array} \right\}$$

Interpretation:

- The robot r is now carrying container c
- The top of pile p is now c' (the container that was below c)
- The position of container c is now the robot r

5.3 Action Applicability

Definizione 5.3 (Action Applicability)

A ground action a is **applicable** (or **executable**) in state s id all preconditions of a are satisfied in s:

$$a \text{ applicable in } s \quad \Leftrightarrow \quad s \models \text{Pre}(a)$$

5.3.1 Examples

Consider two ground actions:

Example 1: $a_1 = \texttt{take}(r_1, c_1, c_2, p_1, d_1)$

In the initial state s_0 where:

- $\texttt{loc}(r_1) = d_1$
- $\texttt{at}(p_1, d_1)$
- $\texttt{pos}(c_1) = p_1$
- $\texttt{on}(c_1) = c_2$
- $\texttt{top}(c_1) = c_1$

- $\texttt{cargo}(r_1) = \texttt{nil}$

All preconditions are satisfied, so a_1 is **executable/applicable** in s_0.

Example 2: $a_2 = \texttt{take}(r_2, c_3, \texttt{nil}, p_4, d_3)$

This action requires:

- $\texttt{loc}(r_2) = d_3$
- $\texttt{at}(p_4, d_3)$
- $\texttt{pos}(c_3) = p_4$
- $\texttt{top}(p_4) = c_3$

However, in s_0, we have $\texttt{pos}(c_3) = p_2$ (not p_4) and $\texttt{loc}(r_2) = d_2$ (not d_3). Therefore, a_2 is **not executable/not applicable** in s_0.

6 State Transition Function

6.1 Formal Definition

The **state transition function** γ defines how actions transform states:

$$\gamma : \text{State} \times \text{Action} \rightarrow \text{State}$$

where $\gamma(s, a)$ computes the successor state obtained by executing ground action a in state s.

6.2 Computing the Successor State

Given a state s and a ground action a (which is a ground instance of an action schema α), the successor state $\gamma(s, a) = s'$ is defined if and only if:

1. $s \models p$ for all $p \in \text{Pre}(a)$ (all preconditions are satisfied)
2. s' is consistent (the resulting state contains no contradictions)

The successor state s' is computed as:

$$s' = \{q \mid q \in \text{Eff}(a)\} \cup \{q \mid s \models q, \neg q \notin \text{Eff}(a), q \notin \text{Eff}(a)\}$$

Interpretation: The new state consists of:

- All literals in the effects of a
- All literals from s that are not affected by a (neither the literal nor its negation appears in the effects)

6.3 Goal States

While the initial state is a specific assignment, goals are typically specified more flexibly.

Definizione 6.1 (Goal Formula)

A **goal formula** G is a conjunction of literals specifying desired properties of goal states.

Definizione 6.2 (Goal States)

The set of **goal states** is:

$$S_g = \{s \mid s \models G\}$$

That is, all assignments that satisfy the goal formula G.

6.4 Comparing Representations: Simplified DWR example

Let us compare the classical and state variable representations using a simplified version of the DWR domain with three actions schema

6.4.1 Classical Representation

In the classical representation, we use propositional atoms to represent facts:

Action 1: $\texttt{take}(r, c, l)$ - robot r takes container c at location l

$$\begin{aligned} &\text{Pre}: && \texttt{loc}(r, l) \wedge \texttt{pos}(c, l) \wedge \neg\texttt{loaded}(r) \\ &\text{Eff}: && \texttt{pos}(c, r) \wedge \neg\texttt{pos}(c, l) \wedge \texttt{loaded}(r) \end{aligned}$$

Action 2: $\texttt{move}(r, l, m)$ - robot r moves from location l to location m

$$\begin{aligned} &\text{Pre}: && \texttt{loc}(r, l) \\ &\text{Eff}: && \texttt{loc}(r, m) \wedge \neg\texttt{loc}(r, l) \end{aligned}$$

Action 3: $\texttt{put}(r, c, l)$ - robot r puts down container c at location l

$$\begin{aligned} &\text{Pre}: && \texttt{loc}(r, l) \wedge \texttt{pos}(c, r) \\ &\text{Eff}: && \texttt{pos}(c, l) \wedge \neg\texttt{pos}(c, r) \wedge \neg\texttt{loaded}(r) \end{aligned}$$

6.4.2 State Variable Representation

In the state variable representation, we use functional notation with assignments:

Action 1: $\texttt{take}(r, c, l)$

$$\begin{aligned} &\text{Pre}: && \texttt{loc}(r) = l \wedge \texttt{pos}(c) = l \wedge \neg\texttt{loaded}(r) \\ &\text{Eff}: && \texttt{pos}(c) \leftarrow r \wedge \texttt{loaded}(r) \end{aligned}$$

Action 2: $\texttt{move}(r, l, m)$

$$\begin{aligned} &\text{Pre}: && \texttt{loc}(r) = l \\ &\text{Eff}: && \texttt{loc}(r) \leftarrow m \end{aligned}$$

Action 3: $\texttt{put}(r, c, l)$

$$\begin{aligned} &\text{Pre}: && \texttt{loc}(r) = l \wedge \texttt{pos}(c) = r \\ &\text{Eff}: && \texttt{pos}(c) \leftarrow l \wedge \neg\texttt{loaded}(r) \end{aligned}$$

6.4.3 Key Differences

- **Classical**: Requires explicit deletion of old values (e.g., $\neg\texttt{loc}(r, l)$ when moving)
- **State Variable**: Implicit deletion through reassignment (e.g., $\texttt{loc}(r) \leftarrow m$ automatically removes the old location)
- **Compactness**: State variables are more concise for multi-valued attributes

7 Planning as Satisfiability (SAT)

7.1 Recap: Core Concepts

Before introducting the SAT encoding, let us recap the fundamental concepts:

- **State**: A Boolean assignment to propositional (ground) literals
- **Action**: Characterized by:
  - **Preconditions**: A set (or conjunction) of literals
  - **Effects**: A set (or conjuction) of literals

7.2 Temproal Instantiation of Variables

To encode planning problems as SAT, we introduce a temporal dimensio to our propositional variables.

Definizione 7.1 (Temporal Instantiation)

Let $\mathcal{X}$ be the set of propositional variables describing the domain. The **temporal instantiation** of $\mathcal{X}$ at time t is:

$$\mathcal{X}@t = \{x@t \mid x \in \mathcal{X}\} \qquad \text{for } t \geq 0$$

where $x@t$ is a propositional variable representing the truth value of x at time step t.

Intuition: Since a plan is a sequence of actions $(a_0, a_1, a_2, \ldots, a_{n-1})$ of length n, we need to track truth value of each proposition at each time step from 0 to n.

7.3 The Bounded Planning Problem

Definizione 7.2 (Bounded Planning Problem)

Given a planning problem $P = \langle \Sigma, I, G \rangle$, the **bounded planning problem** asks:

Does there exists a plan solving P with length exactly n?

This is a decision problem parametrized by the plan length n. RIPRENDERE APPUNTI FINO A SLIDE 46

8 Conflict-Driven Clause Learning (CDCL)

Modern SAT solvers use **Conflict -Driven Clause Learning (CDCL)**, an advanced technique that combines depth-first search with intelligence learning from conflicts to efficently solve large satisfiability problems.

8.1 CDCL Algorithm Overview

CDCL implements **depth-first search strategy** with intelligent backjumping (undo one or more decision) and clause learning:

Definizione 8.1 (CDCL Components)

CDCL combines several key techniques:

- **Unit propagation**: Efficient inference of forced assignments
- **Conflict analysis**: Learning from failures to guide future search
- **Backjumping**: Intelligent backtracking to erliaer decision levels
- **Clause learning**: Adding learned constraints to prevent repeated conflicts

8.2 Backjumping: Generalized Backtracking

Backjumping is a generalization of the basic backtracking (undo one decison) mechanism that allows the solver to jump back to erlier decision levels, not just the immediately preceding one.

Definizione 8.2 (Backjumping vs Backtracking)

In **backjumping**, when a conflict occurs, the solver analyzes the conflict cause to determine the earliest decisione level that contributed to conflict. It then jumps back to that level, rather than just the immediately preceding decision.

Advantage: Backjumping can skip over irrelevant decision levels, significantly reducing the search space compared to basic backtracking.

8.2.1 Conflict analysis example

COnsider a conflict clause $C = L_1 \vee \ldots \vee L_k$ where all literals are currently false. The analysis determines:

- Why each literal L_i is false in the current trail Γ.
- The decision level at which each literal became false
- The erliest decision level that contributed to the conflict

The solver then backjumps to that earliest relevant decision level, effectively skipping decision levels that did not contribute to the conflict.

9 Binary Resolution

The theoretical foundation of conflict analysis relies on **binary resolution**, a foundamental inference rule in propositional logic

9.1 Resolution Rule

Definizione 9.1 (Binary Resolution)

Given two clauses containing complementary literals:

$$\frac{C \vee L \quad D \vee \neg L}{(C \vee D)}$$

The **resolvent** $C \vee D$ is logically entailed by the two premises. This rule allows deriving new clauses that preserve satisfiability.

9.2 Soundness of Resolution

Binary resolution is **sound**, the resolvent is logically entailed by the permise.

Teorema 9.2 (Resolution Soundness)

$IfC \vee L, D \vee \neg L \models RwhereRisanyformula, thenC \vee L, D \vee \neg L \models (C \vee D)$.

Proof. Consider any interpretation v:

- $If v \models L$, then since $v \models D \vee \neg L$, we must have $v \models D$. Since $v \models C \vee L$, we have $v \models C \vee D$.
- $If v \models \neg L$ then since $v \models C \vee L$, we must have $v \models C$. Since $v \models D \vee \neg L$, we have $v \models C \vee D$.

In both cases, $v \models C \vee D$, so the resolvent is logically entailed by the premises.

9.3 Application in Conflict Analysis

In conflict analysis, resolution is used to explain why conflicts occur and to determine which decision levels are responsible:

- When a literal L_k in the conflict clause is false due to propagation (not a decision)
- The propagation was justified by a clause $D \vee \neg L_k$.
- By resolving the conflict clause with the justification clause, we can trace back the decision that ultimately caused the conflict.

10 Explanation Mechanism

The explanation mechenism in CDCL traces the implications that led to a conflict, allowing the solver to learn more general constraints.

Teorema 10.1 (Explanation rule)

$$\langle S \| \Gamma \| C \rangle \rightarrow \langle S \| \Gamma \| C \vee D \rangle$$

if $\neg L \in \Gamma$ where $\neg L$ was propagated using the clause $D \vee \neg L$.

10.1 Explanation Process

The explanation mechanism works as follows:

1. Start with a conflict clause $C = L_1 \vee \ldots \vee L_k$.
2. For each literal L_i that is false due to propagation (not a decision):
   - Find the justification clause that forced $\neg L_i$ into the trail
   - Resolve the current conflict clause with the justification clause
   - Continue untile all literals in the conflict clause are explained by decision
3. The final resolved clause contains only literals that were decided (not propagated)
4. This clause can be learned and added to the formula to prevent similar conflicts

Parte III
SAT-Based Techniques

11 Modern SAT Solving

11.1 CDCL: An evolution of DPLL

CDCL (**Conflict-Driven Clause Learning**) represent an evolution of the DPLL decision procedure for SAT. While CDCL maintains the same basic decision rule

as DPLL, it introduces sophisticated conflict anaysis and learning mechanisms that dramatically improve solver efficiency on real-world instances.

11.1.1 CDCL Transition Rules

The CDCL procedure employs the following transition rules in search mode:

Definizione 11.1 (Decision rule)

$$\langle S \| \Gamma \rangle \to \langle S \| \Gamma, L^d \rangle$$

if $L \notin \Gamma$ and $\neg L \notin \Gamma$, where:

- S is the set of clauses
- Γ is the trail (sequence of assigned literals)
- L^d is a decision literal (chosen by heuristic)

Definizione 11.2 (Propagation rule)

$$\langle S \| \Gamma \rangle \to \langle S \| \Gamma, L_{C \vee L} \rangle$$

if $C \vee L \in S, L \notin \Gamma, \neg L \notin \Gamma$, and $\Gamma \models \neg C$, where:

- C is a disjunction of literals (all false under the current trail)
- L is the unit literal that must be propagated
- The subscript $C \vee L$ indicates the justification clause for the propagation

Definizione 11.3 (Success rule)

$$\langle S \| \Gamma \rangle \to \text{"sat"}$$

if $\Gamma \models S$ (i.e., the trail satisfies all clauses in the set)

Definizione 11.4 (Conflict rule)

$$\langle S \| \Gamma \rangle \to \langle S \| \Gamma \| C \rangle$$

if $C \in S$ and $\Gamma \models \neg C$ (i.e., the trail conflicts with clause C), where C is the conflict clause that triggers transition to conflict analysis mode.

This four rules define the behavior of CDCL in **search mode**, which is identical to the DPLL procedure. The key difference lies in how CDCL handles conflicts through its conflict analysis and learning machanisms.

11.2 Decision Levels in CDCL

The trail Γ in CDCL is organized into **decision levels** that provides structure ti the search process:

- The trail begins at **decision level 0** (empty trail)
- Each decision literal opens a new decision level
- If the maximum decision level in Γ is n, the next decision opens level $n+1$.
- All propagations following a decision remain at the same level untile the next decision

11.2.1 Trail as a Stack Structure

The trail can be conceptualized as a **stack structure**:

1. Initially, the stack is empty (decision level 0)
2. A decision literal is puhsed onto the stack, opening a new level
3. Unit propagations following the decision are pushed onto the stack at the same level
4. Another decision is maade, opening the next level
5. The process continues with alternating decision and propagations
6. During backtracking/backjumping, literals are popped from the stack until reaching the target decision level

This organization enables efficient backjumping by allowing the solver to quickly identify and return to relevant decision levels during conflict analysis.

11.3 Illustrative example

Esempio 11.5 (Instance)

$S = \{P, \neg P \vee Q \vee R, T \vee \neg Q, \neg R \vee Q\}$

The inital propagation and decision sequence proceeds as follows·

$$\langle S \| \epsilon \rangle \xrightarrow{prop} \langle S \| P_P \rangle$$

!!! DA QUI INIZIA LA PARTE DEL GOAT FARINELLI QUINDI RECUPERARE TUTTA LA PARTE PRECEDENTE

Parte IV
Planning Module

12 Motivation

12.1 Deliberative Action

12.1.1 The concept of agent

- Pro-Activity: act to achieve a long term goal

- Rationality: do the right thing
- Adaptation: handle unexpected changes/deviations
- Interaction: other afents, humans.

What is an agent?

12.1.2 Agentic AI

The same concepts, but now the decision and knowledge are based on LLMs and Transfrormers.

What is planning?

It's based on prediction and search, also use a descriptive models of the actions.

- Predict what the actions will do
- Do not tell how to execute actions

Search over **Predicted state** and possible orgaization of feasible actions. Different types of actions $\rightarrow$ different predictive models, planning problems and techniques. 3 types of planning are:

1. Motion planning
2. Perception planning
3. Task planning

12.1.3 Planning and Acting

- Multiple levels of abstraction:
  - Actor are organized into physical sybsystems
  - deliberation resoning techniques

12.1.4 PLanning, Acting and Learning

- Learnining is a very wide concept
  - Train a model vs programming a system
  - Learn a reactive function
  - Learn a partially specified/potentially wrong model
- Key point planning acting and learning should be integrated to relize a robust and effective autonomous system